

# Contexto Completo do Projeto

Sistema dinamico para probabilidades, regras de associacao e programacao linear.

## 1. Contexto do trabalho

O projeto foi desenvolvido a partir do enunciado do professor Jose Carlos Ferreira da Rocha. A proposta era usar uma base com variaveis categoricas para extrair conhecimento probabilistico, representar esse conhecimento como restricoes de um programa linear e permitir que o usuario fizesse perguntas condicionais do tipo  $P(A | B)$ .

## 2. Objetivo implementado

Foi construida uma aplicacao web em Python/Flask com frontend dinamico. O usuario escolhe afirmacoes e uma pergunta na interface. O backend calcula probabilidades, monta o modelo linear, resolve com solver e retorna metricas, intervalo linear, tempo de processamento e conclusao automatica.

## 3. Dataset usado

| Item              | Descricao                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------|
| Arquivo           | data/Crop_recommendation.csv                                                              |
| Origem conceitual | Dataset de recomendacao de culturas agricolas usado como base de solo/cultura.            |
| Colunas           | N, P, K, temperature, humidity, ph, rainfall, label                                       |
| Tratamento        | Colunas numericas sao categorizadas automaticamente; label permanece como classe/cultura. |
| Dinamismo         | O sistema detecta as colunas do CSV e monta a interface com os dominios reais da base.    |

## 4. Arquitetura do sistema

| Camada          | Responsabilidade                                        | Arquivos                               |
|-----------------|---------------------------------------------------------|----------------------------------------|
| Frontend        | Interface para escolher A, B e visualizar resultados.   | index.html, styles.css, script.js      |
| API Python      | Calcula probabilidades, valida consulta e chama solver. | app.py                                 |
| Solver          | Resolve o programa linear via SciPy linprog/HiGHS.      | requirements.txt, app.py               |
| Relatorios      | Gera PDFs e JSONs com explicacoes e saidas calculadas.  | reports/                               |
| Deploy          | Configura Render com Python 3.11.9 e Unicorn.           | render.yaml, Procfile, .python-version |
| Compatibilidade | Alias WSGI para start command antigo do Render.         | your_application/wsgi.py               |

## 5. Fluxo de uso

1. O usuario abre a interface.
2. A interface consulta `/api/metadata` para buscar atributos e valores do dataset.
3. O usuario escolhe o evento A e as afirmacoes B.
4. A interface envia a consulta para `/api/query`.
5. O backend calcula  $P(A)$ ,  $P(B)$ ,  $P(A \text{ e } B)$ , suporte, confianca e lift.
6. O backend monta restricoes lineares e resolve o intervalo de  $P(A | B)$ .
7. A interface mostra resultado, programa linear, tempo e conclusao.

## 6. Matematica do projeto

| Conceito       | Formula                                                     | Uso                                                    |
|----------------|-------------------------------------------------------------|--------------------------------------------------------|
| Marginal       | $P(A) = \text{count}(A) / N$                                | Probabilidade de um valor isolado.                     |
| Conjunta       | $P(A \text{ e } B) = \text{count}(A \text{ e } B) / N$      | Suporte da regra.                                      |
| Condicional    | $P(A   B) = P(A \text{ e } B) / P(B)$                       | Pergunta feita pelo usuario.                           |
| Confianca      | $\text{confidence}(B \rightarrow A) = P(A   B)$             | Precisao da regra.                                     |
| Lift           | $\text{lift} = P(A   B) / P(A)$                             | Forca da associacao.                                   |
| Intervalo      | $\text{round}(p,3) \pm 0,001$                               | Probabilidade intervalar para evitar rigidez numerica. |
| Variavel LP    | $x_w \geq 0$                                                | Probabilidade de cada mundo possivel.                  |
| Normalizacao   | $\text{soma}(x_w) = 1$                                      | Distribuicao de probabilidade valida.                  |
| Restricao      | $L \leq \text{soma}(x_w \text{ onde } E) \leq U$            | Conhecimento probabilistico da base.                   |
| Charnes-Cooper | $P(A \text{ e } B)/P(B) \rightarrow \text{objetivo linear}$ | Permite resolver condicional com linprog.              |

## 7. Solver

O solver nao e separado do projeto. Ele e uma dependencia instalada pelo requirements.txt. O backend importa `scipy.optimize.linprog` e executa o solver dentro da propria aplicacao. No Render, o deploy instala SciPy e Gunicorn; depois inicia o Flask app.

```
scipy.optimize.linprog(method='highs')
```

## 8. Saidas do sistema

| Saida                                       | Conteudo                                                            |
|---------------------------------------------|---------------------------------------------------------------------|
| Interface                                   | Metricas, intervalo linear, programa linear e conclusao automatica. |
| API /api/query                              | JSON dinamico com probabilidades, solver, tempo e conclusao.        |
| relatorio_saida_programacao_linear.pdf      | Graficos, calculos, algoritmos e explicacao.                        |
| saida_calculada_programacao_linear.json     | Calculos numericos dos algoritmos.                                  |
| roadmap_base_cientifica.pdf                 | Roadmap e base cientifica/técnica.                                  |
| conclusao_resultados_programacao_linear.pdf | Conclusao interpretativa dos resultados.                            |
| contexto_completo_projeto.pdf               | Este documento com o contexto total do projeto.                     |

## 9. Tratamento de casos especiais

Quando  $P(B)=0$ , a consulta condicional nao e matematicamente definida, pois haveria divisao por zero. O sistema detecta esse caso antes do solver, explica a inviabilidade e orienta o usuario a reduzir as condicoes ou escolher uma combinacao existente no dataset.

## 10. Deploy no Render

O projeto foi preparado como Web Service Python no Render. A versao Python foi fixada em 3.11.9 para evitar falha de instalacao do SciPy no Python 3.14. O start correto e gunicorn app:app, mas tambem foi criado um modulo `your_application.wsgi` para compatibilidade com start command antigo.

```
Build: python -m pip install --upgrade pip setuptools wheel && python -m pip install -r requirements.txt
Start: gunicorn app:app --bind 0.0.0.0:$PORT --timeout 120
Health check: /healthz
```

## 11. Base cientifica

O projeto se apoia em raciocinio sob incerteza, probabilidades intervalares, regras de associacao, estimativas por frequencia/verossimilhanca e programacao linear. Os links indicados pelo professor foram usados para justificar a formulacao por restricoes, o uso de intervalos, a possibilidade de smoothing e a comparacao futura de solvers.

## 12. Limites e proximos passos

O sistema ja responde consultas condicionais e gera conclusoes. Como proximos passos, e possivel adicionar Laplace smoothing na interface, gerar conjuntas de pares/trios como nivel opcional de restricao, comparar varios solvers, medir curvas de tempo e calcular acuracia quando o atributo perguntado for uma classe.

## 13. Conclusao final

Conclui-se que o projeto implementa o ciclo completo solicitado: extracao de conhecimento probabilistico, transformacao em restricoes lineares, pergunta condicional do usuario, resolucao com solver e apresentacao interpretavel do resultado. A aplicacao tambem registra tempos de processamento, gera relatorios em PDF/JSON e esta pronta para deploy no Render.